

Seminar – User Guide

Topic: T05 – Performance Testing

Subject: Comparative Guide on Grafana k6 vs. Apache JMeter

Course: Software Testing (CS423/CSC13003)

Lecturers: Dr. Lâm Quang Vũ

Instructors / TAs: MSc. Trương Phước Lộc, MSc. Hồ Tuấn Thanh

System Under Test (SUT): EShop (Node.js + SQLite)

Team Members (Team 09):

Student ID	Full Name
23127006	Trần Nguyễn Khải Luân
23127128	Nguyễn Thành Tiến
23127179	Nguyễn Bảo Duy
23127184	Lê Phạm Kiều Duyên

July, 2026

Mục lục

- 1. Giới thiệu
 - 1.1. Problem Space — Vì sao kiểm thử hiệu năng quan trọng với EShop
 - 1.2. Đối tượng hướng đến — k6, JMeter và vai trò của AI
- 2. Cài đặt môi trường (Environment Setup)
 - 2.1. Prerequisites (Tiền điều kiện)
 - 2.2. Installation: Apache JMeter
 - 2.3. Installation: Grafana k6
 - 2.4. Verification & Screenshots (Xác minh cài đặt)
- 3. First Test (End-to-End Test đầu tiên trên EShop)
 - Giai đoạn A: Chuẩn bị chung
 - Giai đoạn B: Nhánh thực thi với Grafana k6
 - Giai đoạn C: Nhánh thực thi với Apache JMeter
- 4. Advanced Usage (Cấu hình nâng cao & Mô hình tải thực tế)
 - 4.1. Chuẩn bị dữ liệu test — Bulk Test Accounts & Token Pool
 - 4.2. Configuration — Workload Model & SLO
 - 4.3. Plugins — Mở rộng công cụ
 - 4.4. Parallel & CLI Runs
 - 4.5. (Tuỳ chọn) Thử nghiệm nâng cao — Tối ưu hoá SQLite
- 5. Troubleshooting (Các lỗi thực tế thường gặp nhất)
 - 5.1. [Lỗi Tool - JMeter] java.net.MalformedURLException: Unrecognized IPV6 address format
 - 5.2. [Lỗi Script - Dynamic Correlation] 401 Unauthorized tại API Cart/Checkout
 - 5.3. [Lỗi Hệ thống - SQLite] SQLITE_BUSY: database is locked khi chạy Spike Test
 - 5.4. [Lỗi Tài nguyên - JMeter] java.lang.OutOfMemoryError khi chạy GUI ở tải lớn
- 6. Failure Modes (Các chế độ lỗi gây hiểu lầm)
 - 6.1. Failure Mode 1: Nhiễm độc "HTTP 200 OK" giả khi Database gặp sự cố (GET /api/products)
 - 6.2. Failure Mode 2: Hiệu ứng sụp đổ hệ thống ảo do thiếu "Think Time" (DDoS giả lập)
 - 6.3. Failure Mode 3: "JWT Ma" sống mãi sau khi khởi động lại Backend
 - 6.4. Failure Mode 4: "Bẫy kiểu dữ liệu" không nhất quán của sản phẩm (Data Type Inconsistency)
- 7. Tham khảo
 - 7.1. Tài liệu môn học và Dự án (Course Materials & Project Sources)
 - 7.2. Tài liệu kỹ thuật chính thức của Công cụ (Official Tool Documentation)
 - 7.3. Tài liệu Kiến trúc và Hệ thống (System Architecture & Engines)
 - 7.4. Trợ lý Trí tuệ Nhân tạo và Công cụ Hỗ trợ (AI Assistants & Generation Tools)

1. Giới thiệu

1.1. Problem Space — Vì sao kiểm thử hiệu năng quan trọng với EShop

Một hệ thống web như EShop không chỉ cần **đúng chức năng** mà còn phải **đứng vững dưới tải thực tế**: hàng chục người dùng lướt sản phẩm cùng lúc, hoặc một đợt flash-sale khiến lượng request tăng vọt trong vài giây. Kiểm thử hiệu năng (Performance Testing) trả lời câu hỏi mà kiểm thử chức năng không thể: *hệ thống chịu được bao nhiêu tải, chịu được trong bao lâu, và nó hỏng theo cách nào khi vượt ngưỡng?* Ba loại kiểm thử được nhóm áp dụng lên EShop phục vụ ba câu hỏi khác nhau:

- **Load Testing:** Hệ thống có đáp ứng đúng SLO (p95 < 500ms, error rate < 1%) ở mức tải kỳ vọng hằng ngày không?
- **Stress Testing:** Đẩy tải vượt xa mức bình thường để tìm điểm gãy cấu trúc (breaking point) và khả năng phục hồi sau đó.
- **Spike Testing:** Mô phỏng một cú tăng tải đột ngột (50 → 500 VU trong 30 giây, kiểu Black Friday) để xem hệ thống có ổn định tức thời hay sụp đổ dây chuyền.

Điểm đặc biệt của EShop nằm ở sự kết hợp giữa Node.js xử lý bất đồng bộ cực nhanh và SQLite vốn là cơ sở dữ liệu dạng file đơn giản. Khi lượng người dùng tăng đột biến trong các đợt flash-sale, dù Node.js có thể tiếp nhận hàng trăm yêu cầu cùng lúc, nhưng SQLite sẽ dễ bị nghẽn (quá tải) vì không thể xử lý đồng thời quá nhiều yêu cầu ghi dữ liệu (như khi thanh toán). Điều này đòi hỏi chúng ta phải mô hình hóa hành vi người dùng thật chuẩn xác để tìm ra giới hạn thực tế của hệ thống.

1.2. Đối tượng hướng đến — k6, JMeter và vai trò của AI

- **k6 (công cụ chính):** Dùng JavaScript — cùng ngôn ngữ với backend EShop — nên dễ viết, dễ đọc và tích hợp tốt vào quy trình phát triển. Engine nhẹ giúp giảm lập hành vi người dùng (think-time, pacing) chính xác, tránh tạo tải giả tạo.
- **JMeter (công cụ dự phòng):** Giao diện kéo-thả (GUI) trực quan giúp nhanh chóng dựng lại kịch bản kiểm thử khi cần đối chiếu kết quả, là phương án an toàn trong buổi demo trực tiếp.
- **AI (ChatGPT/Claude):** Đóng vai trò trợ lý tăng tốc — giúp chuyển đổi nhanh file HAR/log thô thành mã nguồn kiểm thử và gợi ý cấu hình thực tế, để con người tập trung vào khâu kiểm duyệt và tối ưu hoá kịch bản (Human Engineering Audit).

2. Cài đặt môi trường (Environment Setup)

2.1. Prerequisites (Tiền điều kiện)

Trước khi cài JMeter hay k6, máy bạn cần sẵn 2 thứ nền tảng:

- **Java JRE/JDK** — bắt buộc vì JMeter chạy trên nền Java.
- **Node.js** — cần để tự chạy EShop (backend + frontend) làm mục tiêu kiểm thử.

Mở Terminal (macOS/Linux) hoặc Command Prompt/PowerShell (Windows) và gõ 2 lệnh sau để kiểm tra xem máy đã có sẵn chưa:

```
java -version
```

```
node -v
```

Nếu cả hai đều trả về số phiên bản, bạn có thể bỏ qua bước cài Java bên dưới. Nếu báo lỗi "command not found", cài theo hướng dẫn dưới đây tương ứng với hệ điều hành của bạn.

Cài Java — Windows:

Tải và cài đặt trình cài đặt .msi từ [Adoptium \(Temurin\)](#). Trong lúc cài, nhớ tick chọn "Set JAVA_HOME" và "Add to PATH" để dùng được java trực tiếp trong Command Prompt.

Cài Java — macOS:

```
brew install openjdk
```

Cài Java — Linux (Ubuntu/Debian):

```
sudo apt update
```

```
sudo apt install default-jdk -y
```

[Mẹo] Node.js nên cài qua nodejs.org (bản LTS) trên Windows/macOS, hoặc `brew install node` (macOS) / `sudo apt install nodejs npm -y` (Linux) — vì EShop cần Node.js để chạy, không liên quan trực tiếp tới JMeter/k6.

2.2. Installation: Apache JMeter

A. Trên Windows

1. Tải bản `.zip` từ [trang chủ Apache JMeter](#).
2. Giải nén file `.zip` vừa tải (chuột phải → Extract All).
3. Mở thư mục vừa giải nén → vào thư mục `bin` → double-click file `jmeter.bat` để khởi động giao diện JMeter.

[Lưu ý] Windows Explorer đôi khi chỉ "mở xem" file `.zip` (không thực sự giải nén). Luôn dùng "Extract All..." để có một thư mục thật trước khi chạy `jmeter.bat`.

B. Trên macOS

Cách nhanh nhất — dùng Homebrew:

```
brew install jmeter
```

Sau khi cài xong, gõ `jmeter` ở bất kỳ đâu trong Terminal là giao diện JMeter sẽ mở lên.

Ngoài ra, bạn cũng có thể tải bản `.tgz` thủ công từ trang chủ Apache JMeter rồi chạy:

```
tar -xvf apache-jmeter-*.tgz
```

```
cd apache-jmeter-*/bin
```

```
chmod +x jmeter.sh
```

```
./jmeter.sh
```

[💡 Mẹo] Nếu giải nén thủ công, file `jmeter.sh` có thể chưa có quyền thực thi — luôn chạy `chmod +x jmeter.sh` trước khi gọi `./jmeter.sh`, nếu không hệ thống sẽ báo lỗi "Permission denied".

C. Trên Linux (Ubuntu/Debian)

Cách nhanh nhất — cài thẳng qua `apt`:

```
sudo apt install jmeter -y
```

Hoặc nếu muốn dùng đúng bản mới nhất từ Apache (khuyến nghị cho buổi demo chính thức, vì bản trong kho `apt` có thể cũ hơn):

```
tar -xvf apache-jmeter-*.tgz
```

```
cd apache-jmeter-*/bin
```

```
chmod +x jmeter.sh
```

```
./jmeter.sh
```

2.3. Installation: Grafana k6

A. Trên Windows

Dùng winget (có sẵn trên Windows 10/11):

```
winget install k6 --source winget
```

Hoặc nếu bạn dùng Chocolatey:

```
choco install k6
```

B. Trên macOS

```
brew install k6
```

C. Trên Linux (Ubuntu/Debian)

k6 chưa có sẵn trong kho **apt** mặc định, nên cần đăng ký repository chính thức của Grafana trước. Chạy **lần lượt từng lệnh sau** (không gộp chung để tránh lỗi copy-paste):

```
curl -fsSL https://dl.k6.io/key.gpg | sudo gpg --dearmor -o /usr/share/keyrings/k6-archive-keyring.gpg
```

```
echo "deb [signed-by=/usr/share/keyrings/k6-archive-keyring.gpg] https://dl.k6.io/deb stable main" |  
sudo tee /etc/apt/sources.list.d/k6.list
```

```
sudo apt-get update
```

```
sudo apt-get install k6
```

[Lưu ý] Cả 4 lệnh trên đều cần quyền **sudo**. Nếu lệnh thứ 2 (**echo ... | sudo tee ...**) báo lỗi "Permission denied", chắc chắn bạn đã gõ đúng **sudo tee** (không phải chỉ **tee**) — lỗi này rất phổ biến khi dùng pipe **|** với **sudo**.

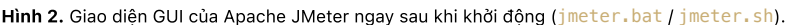
2.4. Verification & Screenshots (Xác minh cài đặt)

Mở Terminal/Command Prompt và gõ lần lượt:

```
jmeter -v
```

Nếu cả 2 lệnh đều trả về số phiên bản (không báo lỗi "command not found"), môi trường của bạn đã sẵn sàng để sang bước tiếp theo.

Hình 1. Terminal hiển thị kết quả thành công của `k6 version` và `jmeter -v`



3. First Test (End-to-End Test đầu tiên trên EShop)

Bài test đầu tiên mô phỏng một **Chained User Journey** gồm 6 bước gọi API liên tiếp, thay vì gọi lẻ tẻ từng endpoint:

Browse → Search → View Detail → Login → Add to Cart → Checkout

Test lẻ tẻ từng API (ví dụ chỉ gọi `GET /api/products` lặp lại) không phản ánh đúng hành vi người dùng thật, và quan trọng hơn — **không thể phát hiện lỗi liên quan đến trạng thái đăng nhập**, vốn chỉ xuất hiện khi các request được gọi đúng thứ tự phụ thuộc nhau (Dynamic Correlation).

[Lưu ý] EShop xác thực phiên đăng nhập bằng **JWT Bearer Token** gửi qua header **Authorization** (đọc trực tiếp từ code `eshop-sut/backend/server.js`), **không dùng cookie**. `GET /api/products`, `/api/products?search=...`, `/api/products/:id` là API công khai (không cần token), nhưng `POST /api/cart` và `POST /api/checkout` **bắt buộc** phải có token hợp lệ — nếu thiếu, cả 2 tool đều nhận lỗi `401 Unauthorized` ngay lập tức. Vì vậy bước **Login phải được thực hiện trước** 2 request ghi dữ liệu này, và token phải được trích xuất thủ công rồi gắn vào header của 2 request đó — ở **cả k6 lẫn JMeter**, không tool nào "tự động" xử lý việc này giúp bạn.

GIẢI ĐOẠN A: CHUẨN BỊ CHUNG

Bước 1: Khởi động backend EShop

```
cd eshop-sut/backend
node server.js
```

Thấy đúng dòng log sau là backend đã sẵn sàng nhận request ở cổng 3000:

```
Connected to database
Database initialized and seeded (Phase 2).
Server is running on http://localhost:3000
```

GIẢI ĐOẠN B: NHÁNH THỰC THI VỚI Grafana k6

Bước 2: Viết phần Login — lấy JWT Token

Tạo file `e2e_test.js` ở gốc dự án. Viết trước phần gọi `POST /api/login` bằng tài khoản có sẵn trong `database.js` (`test@eshop.com / Test1234!`), sau đó đọc trường `token` từ response:

```
import http from "k6/http";
import { check, sleep } from "k6";

export const options = {
  vus: 1,
  iterations: 1,
  thresholds: {
    http_req_failed: ["rate<0.01"],
    http_req_duration: ["p(95)<500"],
  },
};

const BASE_URL = "http://localhost:3000";

export default function () {
  // Bước 2: Login — lấy JWT token
  const loginRes = http.post(
    `${BASE_URL}/api/login`,
    JSON.stringify({ email: "test@eshop.com", password: "Test1234!" }),
    { headers: { "Content-Type": "application/json" } },
  );
  check(loginRes, { "Login - status 200": (r) => r.status === 200 });
  const token = loginRes.json("token");
  sleep(1);
}
```

Bước 3: Viết phần còn lại của kịch bản (Browse → Search → View Detail → Add to Cart → Checkout)

Nối tiếp ngay bên dưới đoạn code ở Bước 2, trong cùng file `e2e_test.js`:

```

const authHeaders = {
  headers: {
    "Content-Type": "application/json",
    Authorization: `Bearer ${token}`,
  },
};

// Browse Home
let res = http.get(`${BASE_URL}/api/products`);
check(res, { "Browse - status 200": (r) => r.status === 200 });
sleep(1);

// Search Products
res = http.get(`${BASE_URL}/api/products?search=iphone`);
check(res, { "Search - status 200": (r) => r.status === 200 });
sleep(1);

// View Product Detail
res = http.get(`${BASE_URL}/api/products/1`);
check(res, { "View Detail - status 200": (r) => r.status === 200 });
sleep(1);

// Add to Cart (cần token)
res = http.post(
  `${BASE_URL}/api/cart`,
  JSON.stringify({ product_id: 1, quantity: 1 }),
  authHeaders,
);
check(res, { "Add to Cart - status 200": (r) => r.status === 200 });
sleep(1);

// Checkout (cần token)
res = http.post(
  `${BASE_URL}/api/checkout`,
  JSON.stringify({
    total_amount: 30000000,
    shipping_address: "123 Đường ABC, Q.1, TP.HCM",
  }),
  authHeaders,
);
check(res, { "Checkout - status 200": (r) => r.status === 200 });
}

```

Bước 4: Chạy kịch bản k6

```
k6 run e2e_test.js
```

Bước 5: Đọc hiểu kết quả k6

Nhìn vào bảng ASCII summary in ra ở cuối terminal, tập trung vào dòng:

```
checks_succeeded.....: 100.00% 6 out of 6
```

Nếu con số này đúng 100% và cả 2 threshold (`http_req_failed`, `http_req_duration`) đều hiện dấu ✓, toàn bộ 6 bước — kể cả Login và 2 request cần token — đã chạy đúng.

[Mẹo] k6 **không** có cơ chế nào "tự động" giữ đăng nhập giữa các request. Việc `token` ở Bước 2 tiếp tục dùng được ở Bước 3 hoàn toàn là do bạn viết chung trong 1 hàm JS và tái sử dụng biến `token` — đây chính là kỹ thuật **Dynamic Correlation** thủ công, không phải phép màu của tool.

```

● (base) tiennt@MacBook-Pro-cua-Tipie seminar % k6 run e2e_test.js



execution: local
script: e2e_test.js
output: -

scenarios: (100.00%) 1 scenario, 1 max VUs, 10m30s max duration (incl. graceful stop):
  * default: 1 iterations shared among 1 VUs (maxDuration: 10m0s, gracefulStop: 30s)

■ THRESHOLDS

http_req_duration
✓ 'p(95)<500' p(95)=26.09ms

http_req_failed
✓ 'rate<0.01' rate=0.00%

■ TOTAL RESULTS

checks_total.....: 6          1.184296/s
checks_succeeded...: 100.00% 6 out of 6
checks_failed.....: 0.00%    0 out of 6

✓ Login - status 200
✓ Browse - status 200
✓ Search - status 200
✓ View Detail - status 200
✓ Add to Cart - status 200
✓ Checkout - status 200

HTTP
http_req_duration.....: avg=8.21ms min=1.41ms med=3.35ms max=32.47ms p(90)=19.7ms p(95)=26.09ms
  { expected_response:true }...: avg=8.21ms min=1.41ms med=3.35ms max=32.47ms p(90)=19.7ms p(95)=26.09ms
http_req_failed.....: 0.00% 0 out of 6
http_reqs.....: 6          1.184296/s

EXECUTION
iteration_duration.....: avg=5.06s min=5.06s med=5.06s max=5.06s p(90)=5.06s p(95)=5.06s
iterations.....: 1          0.197383/s
vus.....: 1          min=1          max=1
vus_max.....: 1          min=1          max=1

NETWORK
data_received.....: 3.4 kB 668 B/s
data_sent.....: 1.1 kB 223 B/s

running (00m05.1s), 0/1 VUs, 1 complete and 0 interrupted iterations
default ✓ [=====] 1 VUs 00m05.1s/10m0s 1/1 shared iters
○ (base) tiennt@MacBook-Pro-cua-Tipie seminar %

```

Hình 3. Kết quả chạy k6 thành công với 6/6 checks pass.

GIẢI ĐOẠN C: NHÁNH THỰC THI VỚI Apache JMeter

Bước 6: Khởi động JMeter và tạo Thread Group

Mở JMeter GUI (jmeter.sh/jmeter.bat) → chuột phải vào **Test Plan** → **Add** → **Threads (Users)** → **Thread Group**. Cấu hình tối giản:

Trường	Giá trị
Number of Threads (users)	1
Loop Count	1

Bước 7: Thêm HTTP Header Manager — khai báo Content-Type (bắt buộc, làm trước khi thêm Sampler)

Trước khi thêm bất kỳ HTTP Request nào, chuột phải vào Thread Group → **Add** → **Config Element** → **HTTP Header Manager**. Nhấn nút **Add** ở bên dưới và điền:

Name	Value
Content-Type	application/json

[⚠️ **Lưu ý quan trọng**] Backend EShop chạy trên Express/Node.js và bắt buộc phải nhận đúng header **Content-Type: application/json** thì mới đọc được dữ liệu (**req.body**) của các request **POST**. Nếu bỏ qua bước này, **cả 6 Sampler sẽ đồng loạt trả về lỗi 400 Bad Request** ngay khi chạy — kể cả với các Sampler **GET** (Browse, Search, View Detail) tuy không có body nhưng vẫn nên để chung 1 Header Manager ở cấp Thread Group cho gọn.

Đặt Header Manager này ở đây (trước Bước 8) vì Sampler **Login** ở bước kế tiếp cũng là một request **POST** — nếu thiếu **Content-Type**, Login sẽ thất bại và không có **token** nào để JSON Extractor trích xuất ở Bước 9.

Bước 8: Thêm 4 HTTP Request Sampler đầu tiên (Browse, Search, View Detail, Login)

Chuột phải vào Thread Group → **Add** → **Sampler** → **HTTP Request**, lặp lại 4 lần. Với **mỗi** Sampler, điền tách riêng từng ô theo đúng bảng dưới — **không** gộp domain và port chung vào 1 ô:

#	Tên Sampler	Protocol	Server Name or IP	Port Number	Path	Body
1	Browse Home	http	localhost	3000	/api/products	—
2	Search Products	http	localhost	3000	/api/products?search=iphone	—
3	View Product Detail	http	localhost	3000	/api/products/1	—
4	Login	http	localhost	3000	/api/login	{"email":"test@eshop.com","password":"Test1234!"}

[⚠️ **Lưu ý quan trọng**] Đây là lỗi phổ biến nhất khi mới dùng JMeter: điền nhầm cả **localhost:3000** vào chung ô **Server Name or IP** sẽ khiến JMeter báo lỗi **java.net.MalformedURLException: Unrecognized IPv6 address format**. Ô **Server Name or IP** chỉ được chứa **localhost** (hoặc **127.0.0.1**) — **tuyệt đối không** kèm dấu **:** và số cổng ở đây; số cổng phải nằm riêng trong ô **Port Number**.

Với Sampler **Login**, khi điền Body ở tab **Body Data**, nhớ đúng cú pháp JSON chuẩn: dùng dấu ngoặc kép **"** cho cả key lẫn value (không dùng dấu nháy đơn **'**), và không để dư dấu phẩy ở cuối — JSON sai cú pháp cũng khiến Express trả lỗi **400 Bad Request** dù đã có Header Manager ở Bước 7.

Bước 9: Thêm JSON Extractor trên Sampler "Login" — trích xuất token

Chuột phải vào Sampler **Login** (Bước 8, #4) → **Add** → **Post Processors** → **JSON Extractor**:

Trường	Giá trị
Names of created variables	token
JSON Path expressions	\$.token

Biến **`\${token}`** giờ có thể dùng ở mọi Sampler đứng **sau** Login trong cùng Thread Group.

[**Lưu ý**] Đây là bước thay thế cho **HTTP Cookie Manager** mà nhiều bản hướng dẫn JMeter mặc định gợi ý — EShop **không dùng cookie**, nên thêm Cookie Manager ở đây sẽ không có tác dụng gì. Nếu bỏ qua bước JSON Extractor này, Sampler Add to Cart/Checkout ở Bước 10 sẽ nhận lỗi **401 Unauthorized**, không phải lỗi "giỏ hàng trống".

Bước 10: Thêm 2 HTTP Request Sampler còn lại + bổ sung header Authorization

Thêm 2 Sampler cuối của journey (cùng quy tắc field Protocol/Server Name/Port/Path tách riêng như Bước 8), đặt sau Sampler Login:

#	Tên Sampler	Protocol	Server Name or IP	Port Number	Path	Body
5	Add to Cart	http	localhost	3000	/api/cart	{"product_id":1,"quantity":1}
6	Checkout	http	localhost	3000	/api/checkout	{"total_amount":30000000,"shipping_address":"123 Đường ABC, Q.1, TP.HCM"}

Quay lại **HTTP Header Manager** đã tạo ở Bước 7, nhấn **Add** thêm 1 dòng mới (giữ nguyên dòng **Content-Type** cũ):

Name	Value
Authorization	Bearer \${token}

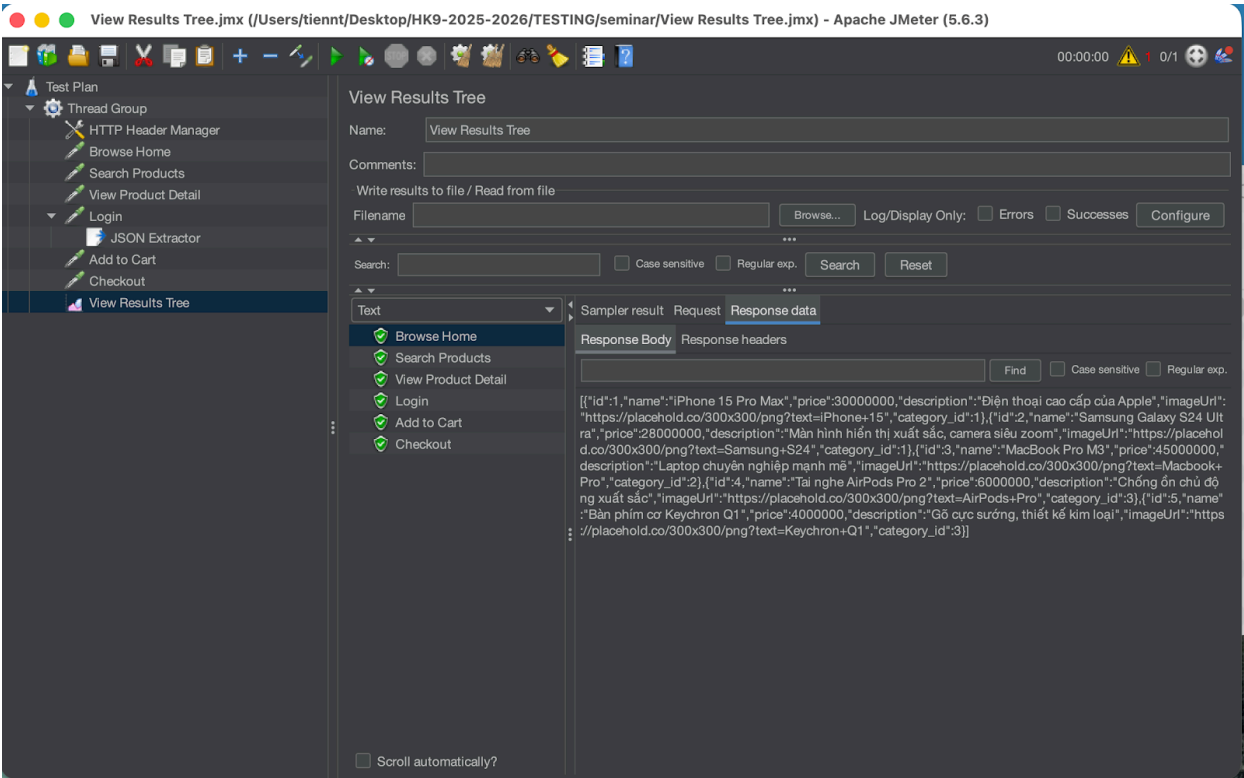
Bước 11: Thêm Listener View Results Tree

Chuột phải vào Thread Group → **Add** → **Listener** → **View Results Tree** để quan sát trực quan từng request.

Bước 12: Thực thi và xác nhận kết quả

Nhấn nút **Play** (🔍, màu xanh lá) trên thanh công cụ. Mở **View Results Tree**, kiểm tra cả 6 Sampler đều hiển thị dấu tick xanh lá:

[Lưu ý] Nếu tất cả Sampler đều đỏ với lỗi **400 Bad Request** → kiểm tra lại Header Manager ở Bước 7 (thiếu **Content-Type**) hoặc cú pháp JSON trong Body Data. Nếu chỉ riêng Sampler #5/#6 đỏ với lỗi **401 Unauthorized** → kiểm tra lại JSON Extractor ở Bước 9 (sai **JSON Path**) hoặc dòng **Authorization** ở Bước 10 chưa được thêm đúng vào Header Manager.



Hình 4. Kết quả chạy **JMeter** thành công — toàn bộ 6 **Sampler** đều **pass** trong **View Results Tree**.

4. Advanced Usage (Cấu hình nâng cao & Mô hình tải thực tế)

Phần First Test ở Section 3 chỉ xác nhận **tính đúng đắn** của kịch bản (1 VU, chạy hết 6 bước, đúng logic). Section này đưa kịch bản đó lên **đúng quy mô tải đã cam kết trong Proposal**: Baseline 50 VU / 5 phút và Spike 50→500 VU / 30 giây, với mô hình tải hỗn hợp (mixed-workload) 60/30/10 thay vì mọi VU đều đi hết trọn vẹn 6 bước như ở First Test.

4.1. Chuẩn bị dữ liệu test — Bulk Test Accounts & Token Pool

[⚠ Lưu ý quan trọng] Backend EShop lưu giỏ hàng bằng `const userCarts = {}` (`server.js`, dòng 14) — một object **JavaScript thuần**, khoá theo `user.id` giải mã từ JWT token. Nếu 500 VU cùng đăng nhập chung 1 tài khoản `test@eshop.com` (như ở First Test) để tiết kiệm công sức, cả 500 VU sẽ **cùng ghi vào đúng 1 mảng giỏ hàng** — không phải lỗi hiệu năng, mà là lỗi mô hình dữ liệu test: kết quả đo được không còn đại diện cho "500 người dùng độc lập" nữa.

Giải pháp: Token Pool Pattern — đăng ký trước N tài khoản test ($N \geq$ số VU tối đa dự kiến, tức $N \geq 500$, để không có VU nào phải dùng lại token của VU khác), đăng nhập lấy sẵn token **một lần duy nhất trước khi đo tải**, rồi phân phối cho từng VU.

Với k6 — dùng hàm `setup()`

`setup()` chỉ chạy **đúng 1 lần** trước khi các stage tải bắt đầu, và **không tính vào metrics đo tải** — đây chính là nơi lý tưởng để "hâm nóng" 500 tài khoản mà không làm nhiễu số liệu Spike Test:

```
const BASE_URL = "http://localhost:3000";
const NUM_TEST_USERS = 500; // >= số VU tối đa (500) để không VU nào trùng token

export function setup() {
  const tokens = [];
  for (let i = 1; i <= NUM_TEST_USERS; i++) {
    const email = `loadtest_user_${i}@eshop.com`;
    const password = "LoadTest123!";

    http.post(
      `${BASE_URL}/api/register`,
      JSON.stringify({ name: `Load Test User ${i}`, email, password }),
      { headers: { "Content-Type": "application/json" } },
    );

    const loginRes = http.post(
      `${BASE_URL}/api/login`,
      JSON.stringify({ email, password }),
      { headers: { "Content-Type": "application/json" } },
    );
    tokens.push(loginRes.json("token"));
  }
  return { tokens }; // giá trị trả về của setup() sẽ được truyền vào default(data)
}

export default function (data) {
  const token = data.tokens[__VU % data.tokens.length];
  // ... dùng token cho các request cần Authorization ở mục 4.2
}
```

[💡 Mẹo] 500 lượt register + login tuần tự trong `setup()` có thể mất 30-60 giây tùy máy — đây là chi phí một lần, chạy **trước** khi đồng hồ đo Spike Test bắt đầu, nên không ảnh hưởng tới số liệu `http_req_duration`. Nếu muốn rút ngắn thời gian chờ, có thể thay vòng lặp tuần tự bằng `http.batch()` để gửi nhiều request đăng ký/đăng nhập song song.

Với JMeter — dùng `setUp Thread Group` + `CSV Data Set Config`

JMeter có sẵn 1 loại Thread Group đặc biệt tên **setUp Thread Group**, chạy đúng 1 lần trước Thread Group chính — tương đương `setup()` của k6:

- Chuột phải vào **Test Plan** → **Add** → **Threads (Users)** → **setUp Thread Group**. Đặt Number of Threads = **500** (bằng đúng N).
- Bên trong, thêm 2 Sampler: **Register** (POST `/api/register`, body dùng hàm `${__threadNum}` để tạo email duy nhất: `loadtest_user_${__threadNum}@eshop.com`) và **Login** (POST `/api/login`, cùng email/password vừa đăng ký).
- Thêm **JSON Extractor** trên Sampler Login (JSON Path: `$.token`, biến `token` — giống hệt cách làm ở Section 3, Bước 9).
- Thêm **JSR223 PostProcessor** (Groovy) ngay sau đó để ghi từng token ra file CSV, mỗi dòng 1 token:

```
new File("tokens.csv").append(vars.get("token") + System.lineSeparator())
```

- Ở **Thread Group chính** (nơi chạy bài Spike Test 500 VU ở mục 4.2), thêm **Add** → **Config Element** → **CSV Data Set Config**, trở vào `tokens.csv`, đặt biến là `token`, **Sharing mode: All threads** (mỗi thread/VU đọc đúng 1 dòng token khác nhau).

[⚠️ Lưu ý quan trọng] Nếu số dòng trong `tokens.csv` (N) **ít hơn** số VU đang chạy, JMeter sẽ tự "Recycle on EOF" — quay vòng đọc lại token từ dòng đầu, khiến nhiều VU lại dùng chung 1 token/giờ hàng, tái diễn đúng lỗi đã nêu ở đầu mục 4.1. Luôn đảm bảo N (số tài khoản test) $\geq$ số VU tối đa của bài test.

4.2. Configuration — Workload Model & SLO

Mô hình tài hỗn hợp 60/30/10 từ Proposal được hiện thực hoá thành **phễu xác suất (Workload Funnel)**: mọi VU đều Browse + Search + View Detail, sau đó rẽ nhánh ngẫu nhiên quyết định có đi tiếp hay không — **không phải** 3 nhóm VU tách biệt độc lập, vì trong đời thực, người checkout luôn là người đã browse/search trước đó.

Nhánh	Tỷ lệ	Hành vi
Chỉ Browse + Search + View Detail	60%	Dừng lại, không cần token
Đi tiếp đến Add to Cart (không Checkout)	30%	Cần token

Nhánh	Tỷ lệ	Hành vi
Đi tiếp đến Checkout (bao gồm cả Add to Cart)	10%	Cần token

Với k6 — `options.stages` cho Spike, `Math.random()` cho phễu

```
export const options = {
  scenarios: {
    mixed_workload: {
      executor: "ramping-vus",
      startVUs: 0,
      stages: [
        { duration: "30s", target: 50 }, // ramp lên Baseline
        { duration: "5m", target: 50 }, // giữ Baseline 50 VU / 5 phút
        { duration: "30s", target: 500 }, // Spike: dốc đứng 50 -> 500 VU / 30s
        { duration: "1m", target: 500 }, // giữ đỉnh Spike
        { duration: "30s", target: 0 }, // ramp-down
      ],
    },
  },
  thresholds: {
    http_req_failed: ["rate<0.01"],
    http_req_duration: ["p(95)<500"],
  },
};

const SEARCH_KEYWORDS = ["iPhone", "Samsung", "Macbook", "AirPods", "Keychron"];

export default function (data) {
  const token = data.tokens[__VU % data.tokens.length];
  const authHeaders = {
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
  };

  // 100%: Browse + Search + View Detail
  http.get(`${BASE_URL}/api/products`);
  sleep(1);
  const keyword =
    SEARCH_KEYWORDS[Math.floor(Math.random() * SEARCH_KEYWORDS.length)];
  http.get(`${BASE_URL}/api/products?search=${keyword}`);
  sleep(1);
  http.get(`${BASE_URL}/api/products/1`);
  sleep(1);

  // Phễu xác suất quyết định rẽ nhánh
  const rand = Math.random();
  if (rand < 0.6) {
    return; // 60%: dừng lại ở đây
  }

  http.post(
    `${BASE_URL}/api/cart`,
    JSON.stringify({ product_id: 1, quantity: 1 }),
    authHeaders,
  );
  sleep(1);

  if (rand >= 0.9) {
    // 10% trong tổng số (nằm trong nhánh rand >= 0.9): đi tiếp Checkout
    http.post(
      `${BASE_URL}/api/checkout`,
      JSON.stringify({
        total_amount: 30000000,
        shipping_address: "123 Đường ABC, Q.1, TP.HCM",
      }),
      authHeaders,
    );
  }
}
```

```

}
// 0.6 <= rand < 0.9 (30%): dừng lại sau Add to Cart, không Checkout
}

```

Với JMeter — cảnh báo Ramp-Up Period, dùng Throughput Controller

[⚠️ Lưu ý quan trọng] **Thread Group** mặc định của JMeter **chỉ ramp tuyến tính** (từ 0 lên N trong X giây) — nó **không thể** tạo hình "giữ ổn định 50 VU trong 5 phút rồi phóng thẳng lên 500 VU trong 30 giây". Muốn tạo đúng đường cong Spike này, bắt buộc phải cài plugin **Custom Thread Groups** ở mục 4.3 trước, rồi dùng thành phần **Free-Form Arrivals Thread Group** (hoặc **Ultimate Thread Group**) để khai báo từng đoạn tải giống hệ thống **stages** của k6 ở trên.

Sau khi đã có tải đúng hình Spike, dùng **Throughput Controller** (chế độ **Percent Executions**) để hiện thực phổ 60/30/10, lồng 2 lớp:

1. Trong Thread Group, đặt Sampler Browse/Search/Detail chạy 100% (không bọc trong Throughput Controller nào).
2. Thêm **Throughput Controller #1**, đặt **40** (%) — đại diện nhóm "đi tiếp quá Search" (30% Cart-only + 10% Checkout = 40% tổng). Bên trong đặt Sampler Add to Cart.
3. Trong Throughput Controller #1, thêm tiếp **Throughput Controller #2** (lồng bên trong), đặt **25** (%) — vì Throughput Controller tính % **trên phạm vi cha của nó**, không phải trên tổng: 10% Checkout / 40% (của Controller #1) = 25%. Bên trong đặt Sampler Checkout.

[Mẹo] Đây là lỗi tính toán phổ biến nhất khi dùng Throughput Controller lồng nhau: phần trăm luôn tính **tương đối với controller cha**, không phải trên tổng số Sampler toàn bài test. Nếu đặt nhầm Controller #2 = **10** thay vì **25**, tỷ lệ Checkout thực tế sẽ chỉ còn 4% (10% × 40%) thay vì đúng 10% như thiết kế.

4.3. Plugins — Mở rộng công cụ

Với k6 — không có plugin runtime, chỉ import thư viện qua `handleSummary()`

k6 là một **binary Go duy nhất**, không có cơ chế "cài plugin" lúc chạy như JMeter. Muốn mở rộng khả năng lõi (thêm giao thức mới...) phải dùng **xx6** để biên dịch lại chính binary k6 — một bước build Go, nằm ngoài phạm vi tài liệu này. Để xuất báo cáo HTML trực quan sau khi chạy CLI, cách đúng là **import thư viện k6-reporter** thẳng từ URL ngay trong script, không cần cài đặt gì:

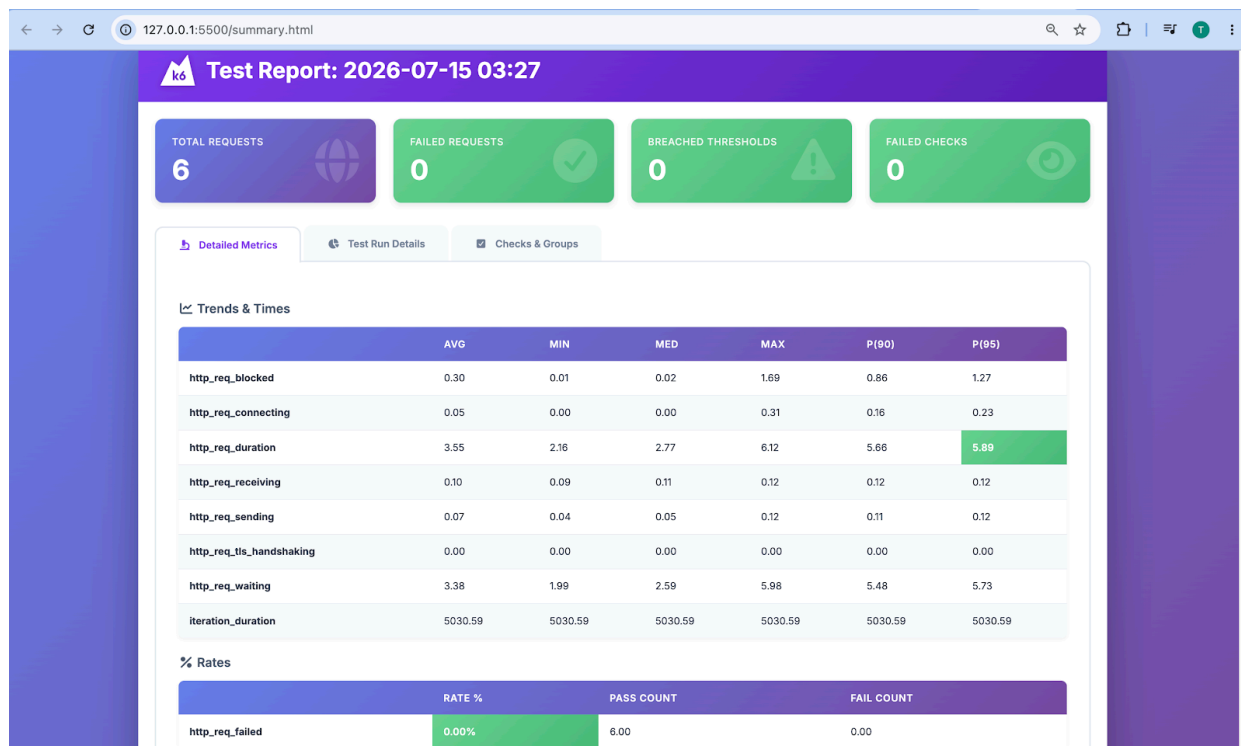
```

import { htmlReport } from "https://raw.githubusercontent.com/benc-uk/k6-reporter/main/dist/bundle.js";

export function handleSummary(data) {
  return {
    "summary.html": htmlReport(data), // xuất báo cáo HTML sau khi test kết thúc
  };
}

```

Chạy lại `k6 run e2e_test.js` như bình thường — file `summary.html` sẽ tự sinh ra ở thư mục hiện tại, mở bằng trình duyệt để xem biểu đồ.



Hình 5. Báo cáo HTML của k6.

Với JMeter — JMeter Plugins Manager

1. Tải `jmeter-plugins-manager.jar` từ [trang chủ jmeter-plugins.org](https://jmeter-plugins.org/).
2. Copy file `.jar` vừa tải vào thư mục `lib/ext/` bên trong thư mục cài JMeter.
3. Khởi động lại JMeter — menu **Options** → **Plugins Manager** sẽ xuất hiện.
4. Trong tab **Available Plugins**, tìm và tick chọn 2 plugin:
 - **Custom Thread Groups** — bắt buộc để tạo đường cong Spike Test ở mục 4.2.
 - **3 Basic Graphs** — vẽ biểu đồ TPS (Transactions per Second) và Response Time trực quan.
5. Nhấn **Apply Changes and Restart JMeter**.

4.4. Parallel & CLI Runs

Với k6 — Multi-Scenario trong cùng 1 tiến trình

k6 chạy nhiều **scenarios** song song **ngay trong cùng 1 tiến trình CLI** (không cần mở nhiều terminal) — chỉ cần khai báo nhiều key trong `options.scenarios` (đã dùng 1 scenario `mixed_workload` ở mục 4.2, có thể thêm scenario khác chạy song song nếu cần mô phỏng nhiều loại traffic cùng lúc).

[Lưu ý] k6 còn hỗ trợ **k6 cloud** để phân tán tải lên hạ tầng Grafana Cloud — đây là dịch vụ **trả phí bên ngoài**, xung đột với nguyên tắc "không tích hợp Grafana Cloud/Prometheus/InfluxDB" đã áp dụng cho dự án. Tài liệu này chỉ nhắc tên để bạn biết là công cụ tồn tại, **không hướng dẫn cấu hình hay chạy thực tế**.

Với JMeter — Bắt buộc Non-GUI Mode cho tải cao

[Lưu ý quan trọng] JMeter GUI được thiết kế để **xây dựng** kịch bản, không phải để **chạy tải lớn**. Ở mức 500 VU, GUI phải vẽ và cập nhật real-time toàn bộ cây Listener (như View Results Tree) cho từng request — bộ nhớ Java Heap sẽ phình to rất nhanh và gần như chắc chắn gặp lỗi `OutOfMemoryError`, làm sập chính công cụ đo trước cả khi SUT kịp bộc lộ giới hạn thật của nó.

Chạy bằng **Non-GUI Mode** qua CLI, xuất thẳng ra Dashboard HTML báo cáo:

```
jmeter -n -t e2e_test.jmx -l results.jtl -e -o report-folder
```

Flag	Ý nghĩa
<code>-n</code>	Chạy ở chế độ Non-GUI
<code>-t</code>	File Test Plan (<code>.jmx</code>) cần chạy

Flag	Ý nghĩa
-l	File log kết quả dạng <code>.jtl</code>
-e -o	Tự sinh Dashboard HTML report vào thư mục chỉ định sau khi chạy xong

4.5. (Tuỳ chọn) Thử nghiệm nâng cao — Tối ưu hoá SQLite

[Lưu ý] Mục này khác biệt với 4 mục trên: nó yêu cầu **sửa trực tiếp code SUT** (`eshop-sut/backend/database.js`), không phải cấu hình công cụ test. Đây là thí nghiệm tuỳ chọn để đối chiếu số liệu, giúp biến giả định kiến trúc đã nêu ở Section 1 (rollback-journal mặc định, không `busy_timeout`) thành bằng chứng thực nghiệm.

Mở `eshop-sut/backend/database.js`, thêm 2 dòng ngay sau khi khởi tạo kết nối (`new sqlite3.Database(...)`), trước khi gọi `initDatabase()`:

```
db.run("PRAGMA journal_mode = WAL;");
db.run("PRAGMA busy_timeout = 5000;");
```

Quy trình đối chiếu trước/sau:

- 1. Chạy bài Spike Test (mục 4.2) trên bản `database.js` gốc (chưa sửa) → ghi lại `http_req_failed` (error rate) và `http_req_duration p(95)` từ summary k6 hoặc Dashboard JMeter.
- 2. Restart backend, thêm 2 dòng PRAGMA ở trên, chạy lại **đúng** bài Spike Test đó (không đổi VU/duration/workload).
- 3. So sánh 2 bộ số liệu:

Chỉ số	Trước (mặc định)	Sau (WAL + busy_timeout)
<code>http_req_failed</code>	Tăng đột ngột khi Cart/Checkout burst — lỗi trả về ngay lập tức (<code>SQLITE_BUSY</code>)	Giảm đáng kể — request chờ thay vì bị từ chối ngay
<code>http_req_duration p(95)</code>	Thấp hơn (vì request lỗi nhanh, không chờ)	Có thể tăng nhẹ (request chờ tới khi có lock thay vì fail ngay)

[Mẹo] Đây chính là bằng chứng thực nghiệm cho nhận định ở Section 1: WAL mode không loại bỏ được giới hạn "1 writer tại 1 thời điểm" của SQLite, nó chỉ **đổi hình dạng thất bại** — từ "báo lỗi ngay" (error rate cao) sang "chờ lâu hơn" (latency cao) khi Checkout burst chạm đúng lúc nhiều VU khác đang ghi.

5. Troubleshooting (Các lỗi thực tế thường gặp nhất)

Mục này gom lại 4 lỗi đại diện cho 4 khía cạnh khác nhau của quá trình kiểm thử EShop — cấu hình tool, viết kịch bản, hệ thống SUT, và cách vận hành tool — mỗi lỗi trình bày theo đúng 1 cấu trúc: Triệu chứng, Nguyên nhân cốt lõi, Cách khắc phục nhanh, Bài học kinh nghiệm.

5.1. [Lỗi Tool - JMeter] `java.net.MalformedURLException: Unrecognized IPv6 address format`

Triệu chứng

Chạy Test Plan trong JMeter, mọi Sampler báo đỏ ngay lập tức (không chờ timeout), Response Code hiện `Non HTTP response code: java.net.MalformedURLException`, Response Message chứa dòng chữ `Unrecognized IPv6 address format`.

Nguyên nhân cốt lõi

Ô **Server Name or IP** trong HTTP Request Sampler bị điền cả domain lẫn cổng chung 1 chỗ, ví dụ `localhost:3000`. JMeter cố phân giải chuỗi này thành địa chỉ, thấy dấu hai chấm nên hiểu nhầm là cú pháp IPv6 (dạng `:::1`) và báo lỗi định dạng.

Cách khắc phục nhanh

Mở lại từng Sampler, tách đúng 3 ô riêng biệt:

Ô cấu hình	Giá trị đúng
Server Name or IP	<code>localhost</code> (hoặc <code>127.0.0.1</code>)
Port Number	<code>3000</code>
Path	<code>/api/products</code> (chỉ phần path, không lặp lại domain/port)

Nếu Test Plan có nhiều Sampler cùng trỏ về `localhost:3000`, thêm 1 lần duy nhất **Add** → **Config Element** → **HTTP Request Defaults** ở cấp Thread Group, điền sẵn Server Name/Port ở đó — các Sampler bên dưới chỉ cần điền Path, tránh gõ lặp lại và tránh gõ sai.

Bài học kinh nghiệm

Đây là lỗi nhập liệu, không phải lỗi mạng hay lỗi backend — thấy `MalformedURLException` là kiểm tra ngay ô Server Name trước, đừng đi debug backend mất thời gian. Xem thêm chi tiết cấu hình field-by-field gốc ở Section 3, Bước 8.

5.2. [Lỗi Script - Dynamic Correlation] 401 Unauthorized tại API Cart/Checkout

Triệu chứng

3 request đầu (Browse, Search, View Detail) đều trả `200 OK`, nhưng đến Add to Cart hoặc Checkout thì nhận `401 Unauthorized`, body trả về `{"error":"Unauthorized"}` hoặc `{"error":"Forbidden"}`.

Nguyên nhân cốt lõi

`POST /api/cart` và `POST /api/checkout` trong `server.js` đều đi qua middleware `authenticateToken`, đọc token từ header `Authorization: Bearer <token>` rồi xác thực bằng `jwt.verify`. Lỗi 401 nghĩa là request gửi đi **không có token** (chưa gọi Login, hoặc quên gắn header); lỗi 403 nghĩa là **có token nhưng sai/hết hạn** (ví dụ trích xuất nhầm field, hoặc token của lần chạy test trước còn sót lại trong biến).

Cách khắc phục nhanh

Với k6: kiểm tra lại đúng 2 dòng trong script — token phải được đọc ra từ response Login và truyền vào header của request sau:

```
const loginRes = http.post(
  `${BASE_URL}/api/login`,
  JSON.stringify({ email, password }),
  {
    headers: { "Content-Type": "application/json" },
  },
);
const token = loginRes.json("token"); // kiểm tra đúng tên field "token"

const res = http.post(
  `${BASE_URL}/api/cart`,
  JSON.stringify({ product_id: 1, quantity: 1 }),
  {
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
  },
);
```

Với JMeter: kiểm tra 2 chỗ theo đúng thứ tự:

- JSON Extractor** gắn trên Sampler Login: **JSON Path expressions** phải là `$.token` (không phải `$.data.token` hay tên khác) và **Names of created variables** phải khớp với biến dùng ở Header Manager.
- HTTP Header Manager**: dòng **Authorization** phải có giá trị đúng `Bearer ${token}` (chú ý chữ hoa/thường và có đúng 1 dấu cách sau "Bearer"), và Header Manager này phải nằm **trước hoặc cùng cấp** với Sampler Cart/Checkout trong cây Test Plan — đặt sau sẽ không có tác dụng.

Bài học kinh nghiệm

EShop dùng JWT Bearer Token, không dùng cookie — không tool nào tự động "nhớ đăng nhập" giúp bạn, việc trích xuất và gắn token luôn phải làm thủ công (kỹ thuật Dynamic Correlation). Xem lại toàn bộ luồng trích xuất/gắn token gốc ở Section 3, Bước 9-10.

5.3. [Lỗi Hệ thống - SQLite] SQLITE_BUSY: database is locked khi chạy Spike Test

Triệu chứng

Chạy thử ở tải nhỏ (1-5 VU) thì pass 100%, nhưng khi lên Spike Test (500 VU) theo cấu hình Section 4.2, console của backend (`node server.js`) hiện hàng loạt dòng lỗi đỏ `SQLITE_BUSY: database is locked`, đồng thời `http_req_failed` trên k6 hoặc tỷ lệ đỏ trên JMeter tăng vọt đúng vào lúc lượng request Checkout dồn dập.

Nguyên nhân cốt lõi

Đã phân tích kỹ ở Section 1 và kiểm chứng bằng thực nghiệm ở Section 4.5: `database.js` mở kết nối SQLite ở chế độ `rollback-journal` mặc định, không có `busy_timeout`. SQLite chỉ cho phép **1 writer tại 1 thời điểm** — khi nhiều request `POST /api/checkout` cùng cố ghi vào bảng `orders` trong lúc Spike, các request tới sau bị từ chối ngay lập tức thay vì chờ.

Cách khắc phục nhanh

Có 2 hướng, làm được ngay không cần sửa code SUT:

- Ở *kịch bản test*: tăng `sleep()` (think-time) giữa các request ghi để giãn tần suất ghi đồng thời — không loại bỏ được giới hạn của SQLite nhưng giảm số lượng va chạm cùng lúc.
- Ở *SUT (tuỳ chọn, đổi hành vi lỗi thay vì giảm tải)*: áp dụng đúng 2 dòng PRAGMA đã hướng dẫn ở Section 4.5 vào `database.js`:

```
db.run("PRAGMA journal_mode = WAL;");
db.run("PRAGMA busy_timeout = 5000;");
```

Bài học kinh nghiệm

`SQLITE_BUSY` không phải lỗi ngẫu nhiên hay lỗi mạng — nó là hệ quả trực tiếp của kiến trúc single-writer đã cảnh báo từ Section 1. Thấy lỗi này xuất hiện đúng lúc tải tăng cao là dấu hiệu công cụ test đang hoạt động đúng: nó vừa phát hiện ra giới hạn thật của hệ thống, không phải lỗi của script.

5.4. [Lỗi Tài nguyên - JMeter] java.lang.OutOfMemoryError khi chạy GUI ở tải lớn

Triệu chứng

Mở sẵn Test Plan trong JMeter GUI, cấu hình Thread Group lên 500 VU rồi nhấn Play trực tiếp trên giao diện. Sau vài giây đến vài chục giây, JMeter phản hồi chậm dần, cuối cùng đứng hình hoặc hiện lỗi `java.lang.OutOfMemoryError: Java heap space` trong log (`bin/jmeter.log`), có thể kèm crash toàn bộ ứng dụng.

Nguyên nhân cốt lõi

JMeter GUI được thiết kế để **xây dựng** kịch bản, không phải để **thực thi tải lớn**. Ở 500 VU, GUI phải đồng thời vẽ và cập nhật real-time toàn bộ cây Listener (như View Results Tree) cho từng request, giữ toàn bộ dữ liệu response trong bộ nhớ Java Heap mặc định (thường chỉ vài trăm MB) — heap phình to rất nhanh và tràn trước cả khi SUT (EShop) kịp bộc lộ giới hạn thật của nó.

Cách khắc phục nhanh

Đóng cửa sổ GUI đang treo (buộc phải tắt tiến trình nếu đứng hình hẳn), sau đó chạy lại đúng Test Plan bằng **Non-GUI Mode** như đã hướng dẫn ở Section 4.4:

```
jmeter -n -t e2e_test.jmx -l results.jtl -e -o report-folder
```

Chỉ dùng GUI để **thiết kế** Test Plan (thêm Sampler, Config Element...) ở tải rất nhỏ (1 VU) để kiểm tra logic đúng trước, sau đó luôn chuyển sang Non-GUI khi chạy thật.

Bài học kinh nghiệm

Quy tắc bất di bất dịch của JMeter: **GUI để thiết kế, Non-GUI để đo tải**. Đây không phải giới hạn của EShop mà là giới hạn của chính công cụ JMeter — nhầm lẫn 2 vai trò này là nguyên nhân phổ biến nhất khiến người mới nghĩ oan rằng backend bị crash trong khi thực chất là JMeter tự làm sập chính nó trước.

6. Failure Modes (Các chế độ lỗi gây hiểu lầm)

Khác với Section 5 (lỗi làm request báo đỏ, dễ nhận ra), 4 chế độ dưới đây nguy hiểm hơn nhiều: **công cụ test báo PASS 100%, Error Rate 0%**, trong khi hệ thống thực chất đang hỏng hoặc trả sai dữ liệu ở phía dưới. Toàn bộ 4 mục đã được đối chiếu trực tiếp với `eshop-sut/backend/server.js` và `database.js`, không suy đoán.

6.1. Failure Mode 1: Nhiễm độc "HTTP 200 OK" giả khi Database gặp sự cố (GET /api/products)

Ảo ảnh kết quả (The Illusion)

k6/JMeter báo cáo thành công 100% (**Error Rate = 0%**), Latency cực thấp (vài ms) trên cả 2 request Browse Home và View Product Detail — nhìn vào dashboard, không ai nghĩ ngờ có bất kỳ vấn đề gì.

Sự thật im lặng (The Silent Reality)

Đối chiếu `server.js`:

```
// dòng 152-156 - GET /api/products (nhánh không có search)
db.all("SELECT * FROM products", [], (err, rows) => {
  res.json(rows); // KHÔNG kiểm tra "err" - nếu lỗi, rows = undefined, vẫn trả về 200
});

// dòng 159-165 - GET /api/products/:id
db.get("SELECT * FROM products WHERE id = ?", [req.params.id], (err, row) => {
  if (!row) return res.status(200).json({}); // lỗi DB cũng rơi vào đây, trả 200 kèm object rỗng
  if (row.id % 2 === 0) row.price = row.price.toString();
  res.json(row);
});
```

Tầng backend hoàn toàn phớt lờ tham số `err` do driver `sqlite3` trả về. Khi SQLite bị khoá cứng (`SQLITE_BUSY`, đã phân tích ở Section 1 và 5.3) đúng lúc 2 route này được gọi, `rows/row` sẽ là `undefined`, nhưng HTTP Status vẫn giữ nguyên `200 OK`. Người dùng thật sẽ thấy trang chủ trống trơn không có sản phẩm nào, còn công cụ test — nếu chỉ check mã trạng thái — vẫn báo PASS hoàn hảo.

Biện pháp kiểm duyệt (The Prevention)

Không bao giờ chỉ check HTTP Status. Luôn quét thêm nội dung Body:

Với k6:

```
const res = http.get(`${BASE_URL}/api/products`);
check(res, {
  "status is 200": (r) => r.status === 200,
  "body là mảng và có ít nhất 1 sản phẩm": (r) =>
    Array.isArray(r.json()) && r.json().length > 0,
});
```

Với JMeter: thêm **Response Assertion** trên Sampler Browse Home, chọn **Field to Test: Text Response, Pattern Matching Rules: Contains**, và điền chuỗi con bắt buộc phải có trong body (ví dụ `"price"`) — nếu body rỗng/`undefined`, Assertion sẽ báo đỏ dù HTTP Status vẫn là 200.

[⚠️ Lưu ý quan trọng] Đây là lý do vì sao "chỉ check status 200" là thói quen nguy hiểm nhất trong Performance Testing — con số Error Rate đẹp trên dashboard không đồng nghĩa với việc SUT đang trả đúng dữ liệu.

6.2. Failure Mode 2: Hiệu ứng sụp đổ hệ thống ảo do thiếu "Think Time" (DDoS giả lập)

Ảo ảnh kết quả (The Illusion)

Chạy Spike Test đốc đúng (500 VU) nhưng quên `sleep()`/Timer giữa các bước, hệ thống báo lỗi kết nối hàng loạt gần như ngay lập tức. Người kiểm thử vội vàng kết luận: "EShop không chịu nổi 500 người dùng thật".

Sự thật im lặng (The Silent Reality)

Rà soát toàn bộ route handler trong `server.js`: không có bất kỳ đoạn code đồng bộ (CPU-bound) nào chặn Event Loop — mọi thao tác DB đều đi qua callback bất đồng bộ của `sqlite3`. Vì vậy, nguyên nhân sập **không nằm ở việc Express "nghe Event Loop"** như suy đoán ban đầu, mà đến từ 2 cơ chế thật:

- Cạnh tranh tài nguyên cục bộ (Client-Server Resource Contention):** k6/JMeter (client) và `node server.js` (SUT) chạy chung 1 máy, tự tranh giành CPU/RAM của chính máy đó — độ trễ đo được phản ánh máy test đang nghẽn, không phải giới hạn thật của server.
- Nghẽn tại Thread Pool của libuv (UV_THREADPOOL_SIZE = 4 mặc định):** driver `sqlite3` (đã xác nhận ở Section 1) đẩy mọi lệnh SQL qua đúng 4 thread nền cố định. Không có pacing, hàng trăm request dồn cục cùng lúc, xếp hàng chờ 4 thread này còn trước cả khi chạm tới lock của SQLite.

Việc thiếu Think Time đã biến bài Spike Test hợp lệ thành một cuộc tấn công dạng DDoS tự gây ra — hệ thống "sập" vì bị dội một lượng request phi thực tế trong tích tắc, không phải vì năng lực nghiệp vụ thật sự của EShop yếu.

Biện pháp kiểm duyệt (The Prevention)

Luôn thiết lập độ trễ mô phỏng hành vi đọc/lướt thật của con người, tối thiểu 1-3 giây giữa các bước:

Với `k6`: `sleep(1)` giữa mỗi request (đã áp dụng xuyên suốt từ Section 3).

Với `JMeter`: `Add` → `Timer` → `Uniform Random Timer`, đặt `Constant Delay Offset: 1000` (ms) và `Random Delay Maximum: 2000` (ms) để mô phỏng độ trễ suy nghĩ dao động 1-3 giây, không cố định tuyệt đối như con người thật.

[Mẹo] Nếu nghi ngờ SUT thật sự quá tải hay chỉ đang bị nghẽn do chạy chung máy, hãy thử tách load generator sang 1 máy khác (hoặc container riêng) và so sánh lại kết quả — nếu error rate giảm mạnh, nguyên nhân gốc là cạnh tranh tài nguyên cục bộ, không phải giới hạn của EShop.

6.3. Failure Mode 3: "JWT Ma" sống mãi sau khi khởi động lại Backend

Ảo ảnh kết quả (The Illusion)

Backend SUT được restart và database reset trắng trơn giữa các lần chạy test (`node server.js` chạy lại `initDatabase()` — xoá sạch bảng `users` cũ), nhưng các request gửi kèm Token cũ do `k6/JMeter` lưu từ phiên trước lên API Checkout vẫn báo PASS (`200 OK`) một cách kỳ lạ.

Sự thật im lặng (The Silent Reality)

Đối chiếu `server.js` dòng 100-110:

```
const authenticateToken = (req, res, next) => {
  const authHeader = req.headers["authorization"];
  const token = authHeader && authHeader.split(" ")[1];
  if (token == null) return res.status(401).json({ error: "Unauthorized" });

  jwt.verify(token, SECRET_KEY, (err, user) => {
    if (err) return res.status(403).json({ error: "Forbidden" });
    req.user = user; // KHÔNG hề truy vấn lại DB xem user.id có còn tồn tại hay không
    next();
  });
};
```

`jwt.verify` chỉ kiểm tra **chữ ký số hợp lệ** — hoàn toàn không truy vấn lại bảng `users` để xác minh `user.id` còn tồn tại thật hay không. Hai yếu tố sau khiến lỗi hổng này càng nghiêm trọng hơn:

- `jwt.sign({ id: user.id, role: user.role }, SECRET_KEY)` (dòng 51) **không có `expiresIn`** → token không bao giờ tự hết hạn.
- `database.js` không bật `PRAGMA foreign_keys = ON` → lệnh `INSERT INTO orders (user_id, ...)` vẫn thành công dù `user_id` đó không còn tồn tại trong bảng `users` vừa bị reset.

Kết quả: một "người dùng ma" — không hề tồn tại trong database hiện tại — vẫn tạo đơn hàng thành công (`200 OK`, có `orderId` hẳn hoi) trên hệ thống mới reset.

Biện pháp kiểm duyệt (The Prevention)

Luôn để kịch bản test thực hiện một luồng **Login mới thực sự ở đầu mỗi phiên chạy** để lấy token khớp với đúng vòng đời hiện tại của database, thay vì tái sử dụng token tĩnh đã lưu từ lần chạy trước (đúng theo pattern Token Pool ở Section 4.1 — token luôn được cấp mới trong `setup()/setUp Thread Group` của phiên chạy hiện tại, không đọc lại file token cũ từ hôm trước).

[⚠️ Lưu ý quan trọng] Nếu thấy Checkout luôn PASS dù vừa restart backend, đừng vội mừng — hãy kiểm tra `orders` table thật (`sqlite3 database.sqlite "SELECT * FROM orders;"`) xem `user_id` đó có khớp với dòng nào trong `users` không. PASS trên tool không đồng nghĩa với dữ liệu hợp lệ trên SUT.

6.4. Failure Mode 4: "Bẫy kiểu dữ liệu" không nhất quán của sản phẩm (Data Type Inconsistency)

Ảo ảnh kết quả (The Illusion)

Toàn bộ request lấy chi tiết sản phẩm (`GET /api/products/:id`) đều trả `200 OK` với đầy đủ JSON hợp lệ — nhìn qua bất kỳ 1 response đơn lẻ nào cũng "trông đúng", không có gì bất thường.

Sự thật im lặng (The Silent Reality)

Đối chiếu `server.js` dòng 162:

```
if (row.id % 2 === 0) row.price = row.price.toString();
```

Backend cố tình ép kiểu `price` thành `String` với sản phẩm có `id` chẵn (ví dụ `id=2, 4`), và giữ nguyên kiểu `Number` với `id` lẻ (`id=1, 3, 5`) — cùng 1 trường `price`, nhưng kiểu dữ liệu trả về **không nhất quán** tùy theo sản phẩm nào được gọi. Một bài test chỉ gọi thử 1-2 sản phẩm rồi khẳng định "API ổn định" sẽ hoàn toàn bị đánh lừa. Ứng dụng client (frontend hoặc app tích hợp) nếu cộng dồn `price` để tính tổng giỏ hàng mà không ép kiểu trước, sẽ ra kết quả sai lệch hoặc lỗi runtime ngay khi gặp đúng sản phẩm ID chẵn.

Biện pháp kiểm duyệt (The Prevention)

Viết Assertion kiểm tra nghiêm ngặt **kiểu dữ liệu**, không chỉ sự tồn tại của trường, và lặp qua nhiều ID khác nhau (cả chẵn lẫn lẻ) thay vì test 1 ID cố định:

Với `k6`:

```
check(res, {
  "status is 200": (r) => r.status === 200,
  "price là kiểu number": (r) => typeof r.json().price === "number",
});
```

Với `JMeter`: dùng **JSR223 Assertion** (Groovy) trên Sampler View Product Detail:

```
def price = new groovy.json.JsonSlurper().parseText(prev.getResponseDataAsString()).price
if (!(price instanceof Number)) {
  AssertionResult.setFailure(true)
  AssertionResult.setFailureMessage("price không phải kiểu Number: " + price)
}
```

[Mẹo] Luôn thiết kế Test Data (danh sách ID sản phẩm dùng để test) bao gồm cả ID chẵn lẫn lẻ — nếu chỉ test với ID=1 (giống hầu hết ví dụ trong tài liệu này), lỗi này sẽ không bao giờ lộ diện.

7. Tham khảo

Toàn bộ đường link bên dưới đã được kiểm tra và xác nhận truy cập được tại thời điểm biên soạn tài liệu (07/2026). Định dạng trích dẫn theo chuẩn APA.

7.1. Tài liệu môn học và Dự án (Course Materials & Project Sources)

- Lâm Quang Vũ. (2026). *Slide bài giảng môn học Kiểm thử phần mềm (Software Testing)*. Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên - ĐHQG-HCM. (Trích dẫn nội dung lý thuyết nền tảng về Load, Stress, Spike Testing).
- Trương Phước Lộc, & Hồ Tuấn Thanh. (2026). *Tài liệu định hướng đề tài Seminar - T05: Performance Testing*. Bộ môn Công nghệ Phần mềm, Trường Đại học Khoa học Tự nhiên - ĐHQG-HCM.
- Nhóm 09 - CS423. (2026). *Mã nguồn hệ thống thử nghiệm EShop (SUT - System Under Test)*. Đường dẫn thư mục: `eshop-sut/backend`. (Dùng để đối chiếu các route API `/api/products`, `/api/login`, `/api/cart`, `/api/checkout` và cấu hình `database.js` xuyên suốt Section 1, 3, 4, 5, 6).

7.2. Tài liệu kỹ thuật chính thức của Công cụ (Official Tool Documentation)

- Grafana Labs. (2026). *Options reference - stages*. k6 Documentation. <https://grafana.com/docs/k6/latest/using-k6/k6-options/reference/> (Cấu hình đường cong tải Spike Test ở Section 4.2).
- Grafana Labs. (2026). *Ramping VUs executor*. k6 Documentation. <https://grafana.com/docs/k6/latest/using-k6/scenarios/executors/ramping-vus/>
- Grafana Labs. (2026). *Checks*. k6 Documentation. <https://grafana.com/docs/k6/latest/using-k6/checks/> (Cơ chế Assertion dùng xuyên suốt Section 3, 5, 6).
- Grafana Labs. (2026). *Test lifecycle*. k6 Documentation. <https://grafana.com/docs/k6/latest/using-k6/test-lifecycle/> (Cơ sở kỹ thuật cho `setup()` trong Token Pool Pattern ở Section 4.1 - tài liệu chính thức cảnh báo rõ: quản lý dữ liệu test không đúng cách trong `setup()` có thể dẫn tới "misleading load test results", đúng như phân tích ở Section 6).
- Apache Software Foundation. (2026). *Component reference: JSON Extractor, HTTP Header Manager, CSV Data Set Config, Throughput Controller*. Apache JMeter User's Manual. https://jmeter.apache.org/usermanual/component_reference.html (Nền tảng kỹ thuật cho Section 3, 4.1, 4.2).
- Apache Software Foundation. (2026). *Getting started - Non-GUI (CLI) mode*. Apache JMeter User's Manual. <https://jmeter.apache.org/usermanual/get-started.html> (Căn cứ cho khuyến nghị bắt buộc dùng CLI ở Section 4.4 và 5.4).

10. JMeter-Plugins.org. (2026). *Ultimate Thread Group*. <https://jmeter-plugins.org/wiki/UltimateThreadGroup/> (Plugin dựng đường cong Spike Test ở Section 4.3).
11. JMeter-Plugins.org. (2026). *Concurrency Thread Group*. <https://jmeter-plugins.org/wiki/ConcurrencyThreadGroup/>
12. TryGhost. (2026). *node-sqlite3: Asynchronous, non-blocking SQLite3 bindings for Node.js* [Mã nguồn và tài liệu]. GitHub. <https://github.com/TryGhost/node-sqlite3/wiki> (Xác nhận cơ chế driver `sqlite3` dùng trong `database.js` chạy qua thread pool, không đồng bộ - căn cứ cho phân tích Section 1 và 6.2).

7.3. Tài liệu Kiến trúc và Hệ thống (System Architecture & Engines)

13. SQLite Consortium. (2026). *Write-Ahead Logging*. SQLite Documentation. <https://sqlite.org/wal.html> (Nền tảng cho thí nghiệm WAL mode ở Section 4.5).
14. SQLite Consortium. (2026). *Pragma statements supported by SQLite - busy_timeout*. SQLite Documentation. <https://sqlite.org/pragma.html> (Giải thích cơ chế `PRAGMA busy_timeout` dùng ở Section 4.5 và 5.3).
15. OpenJS Foundation / Node.js. (2026). *Command-line API - UV_THREADPOOL_SIZE*. Node.js Documentation. <https://nodejs.org/api/cli.html> (Căn cứ kỹ thuật cho giới hạn 4 thread mặc định của libuv, phân tích ở Section 4 và 6.2).
16. Internet Engineering Task Force (IETF). (2015). *RFC 7519: JSON Web Token (JWT)*. <https://www.rfc-editor.org/rfc/rfc7519.html> (Đặc tả kỹ thuật gốc của JWT, dùng để giải thích tính chất Stateless trong phân tích "JWT Ma" ở Section 6.3).
17. Redis. (2026). *JSON Web Tokens (JWT) are dangerous for user sessions*. Redis Engineering Blog. <https://redis.io/blog/json-web-tokens-jwt-are-dangerous-for-user-sessions/> (Phân tích độc lập từ bên thứ ba về đúng rủi ro đã kiểm chứng ở Section 6.3: JWT không tra cứu lại database, không thể vô hiệu hoá tức thời sau khi phát hành).

7.4. Trợ lý Trí tuệ Nhân tạo và Công cụ Hỗ trợ (AI Assistants & Generation Tools)

18. OpenAI. (2026). *ChatGPT (phiên bản GPT-4o / GPT-4)* [Mô hình ngôn ngữ lớn]. <https://chatgpt.com> (Phân tích dữ liệu thô từ log/HAR của EShop, tự động hoá việc dịch và xây dựng khung kịch bản kiểm thử k6 ban đầu).
19. Anthropic. (2026). *Claude (phiên bản Claude 3.5 Sonnet)* [Mô hình ngôn ngữ lớn]. <https://claude.ai> (Hỗ trợ rà soát mã nguồn `server.js` để tìm kiếm các Failure Modes ở Section 6, tối ưu hoá thuật toán phân bổ Token Pool ở Section 4.1, và kiểm toán cấu hình kịch bản).
20. Google. (2026). *Gemini (phiên bản Gemini 1.5 Pro / Flash)* [Mô hình ngôn ngữ lớn]. <https://gemini.google.com> (Hỗ trợ định hình cấu hình so sánh công cụ k6/JMeter, scaffold bố cục báo cáo và chuẩn hoá văn phong kỹ thuật của User Guide).